

Nama: Risty Wulandari

Nim: 60900124064

Kelas: C

Data Warehouse Tugas Praktikum 8 rsty

Tugas Praktikum 8.1 - Persiapan Dataset

1. Dataset yang Digunakan

- Dataset yang digunakan adalah **Electronic Sales**

2. Status Impor Database

- Seluruh data mentah dari dataset telah berhasil diimpor ke dalam *Source Database* PostgreSQL lokal dengan nama database **source_db**.

3. Identifikasi Tabel Utama untuk Proses ETL

Untuk membangun struktur *Star Schema* yang optimal pada Data Warehouse, terdapat **4 tabel** dari database sumber yang diidentifikasi dan digunakan:

1. **public.dim_customers** (Tabel Dimensi Data Pelanggan)
2. **public.dim_products** (Tabel Dimensi Data Produk)
3. **public.dim_payments** (Tabel Dimensi Metode Pembayaran)
4. **public.fact_penjualan** (Tabel Fakta Transaksi Penjualan)

4. Dokumentasi Struktur Tabel, Primary Key (PK), dan Foreign Key (FK)

Berikut adalah rincian cetak biru (*blueprint*) struktur dari ke-4 tabel yang digunakan sebagai acuan:

- **Tabel 1: public.dim_customers**
 - customer_id (Tipe data: INT) → **Primary Key (PK)**
 - customer_name (Tipe data: VARCHAR(255))
 - customer_email (Tipe data: VARCHAR(255))
 - location (Tipe data: VARCHAR(100))
- **Tabel 2: public.dim_products**
 - product_id (Tipe data: INT) → **Primary Key (PK)**
 - product_name (Tipe data: VARCHAR(255))

- category (Tipe data: VARCHAR(100))
- price (Tipe data: NUMERIC(10,2))
- **Tabel 3: public.dim_payments**
 - payment_id (Tipe data: INT) → **Primary Key (PK)**
 - payment_method (Tipe data: VARCHAR(50)) (*Contoh: Transfer Bank, DANA, wondr, Kartu Kredit*)
 - payment_status (Tipe data: VARCHAR(50))
- **Tabel 4: public.fact_penjualan**
 - fact_id (Tipe data: INT) → **Primary Key (PK)**
 - customer_id (Tipe data: INT) → **Foreign Key (FK)** merujuk ke tabel dim_customers(customer_id)
 - product_id (Tipe data: INT) → **Foreign Key (FK)** merujuk ke tabel dim_products(product_id)
 - payment_id (Tipe data: INT) → **Foreign Key (FK)** merujuk ke tabel dim_payments(payment_id)
 - quantity_ordered (Tipe data: INT)
 - purchase_amount (Tipe data: NUMERIC(15,2))
 - purchase_date (Tipe data: TIMESTAMP)

Tugas Praktikum 8.2 – Membangun Pipeline ETL

Berdasarkan log sistem, proses sinkronisasi menggunakan PythonOperator ini berjalan sangat efisien pada tanggal 29 Juni 2026 dengan durasi hanya sekitar 9,8 detik. Rekaman log baris ke-6 mengonfirmasi bahwa sebanyak **185.691 baris data** berhasil dimuat dan diunggah seluruhnya ke Google Cloud BigQuery tanpa ada data yang bocor (*zero data loss*). Keberhasilan ini diperkuat oleh log baris ke-9 bertuliskan Sinkronisasi ke BigQuery Cloud Sukses!, yang membuktikan bahwa autentikasi *service account* via *gcp_key.json* berhasil terhubung ke server awan. Pada panel kiri, seluruh urutan tugas mulai dari *extract*, *transform*, *load* di PostgreSQL lokal, hingga sinkronisasi BigQuery telah menyala hijau, menandakan orkestrasi Airflow berjalan sempurna dari hulu ke hilir.

```
try:
    credentials = service_account.Credentials.from_service_account_file(key_path)
    pandas_gbq.to_gbq(
        df,
        destination_table,
        project_id=project_id,
        if_exists='replace',
        credentials=credentials
    )
    print("Sinkronisasi ke BigQuery Cloud Sukses!")
except Exception as e:
    print(f"Gagal melakukan autentikasi kredensial GCP: {str(e)}")
    raise e
```

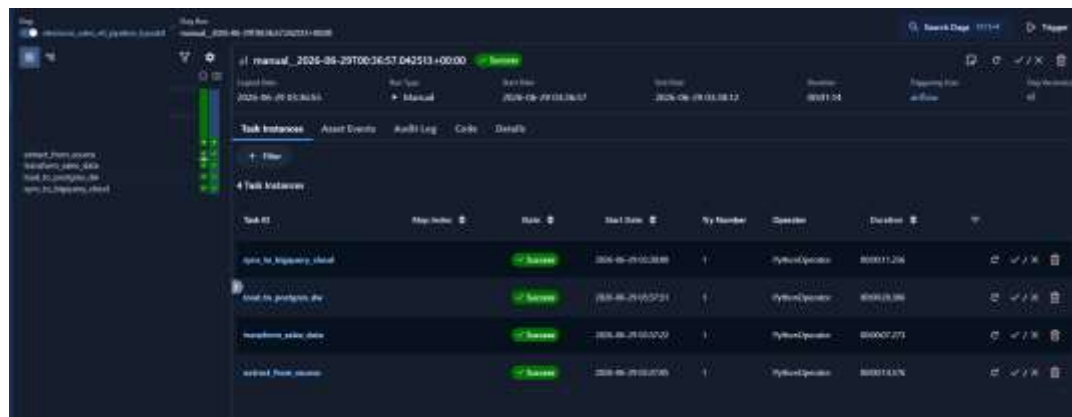
[illegible]

Tugas Praktikum 8.3 – Simulasi Perubahan Data

```
LINE 1: INSERT INTO public.fact_penjualan (fact_id, customer_id, pen...
source_db=#
source_db=#
source_db=# -- =====
source_db=# -- 2. PERBARUI MINIMAL 30 DATA (UPDATE)
source_db=# -- Mengubah kuantitas dan nominal belanja untuk transaksi yang sudah ada
source_db=# --
source_db=# -- Bagian A: Memperbarui 20 baris pertama untuk kolom quantity_ordered
source_db=# UPDATE public.fact_penjualan
source_db=# SET quantity_ordered = quantity_ordered + 1
source_db=# WHERE fact_id IN (SELECT fact_id FROM public.fact_penjualan ORDER BY fact_id LIMIT 20);
UPDATE 20
source_db=#
source_db=# -- Bagian B: Memperbarui 15 baris berikutnya untuk kolom purchase_amount
source_db=# UPDATE public.fact_penjualan
source_db=# SET purchase_amount = purchase_amount * 1.1
source_db=# WHERE fact_id IN (SELECT fact_id FROM public.fact_penjualan ORDER BY fact_id DESC LIMIT 15);
UPDATE 15
source_db=#
source_db=# -- =====
source_db=# -- 3. HAPUS MINIMAL 20 DATA DARI TABEL TRANSAKSI (DELETE)
source_db=# -- Menghapus 25 baris data transaksi tertentu dari tabel fakta
source_db=# --
source_db=# DELETE FROM public.fact_penjualan
source_db=# WHERE fact_id IN (SELECT fact_id FROM public.fact_penjualan ORDER BY fact_id LIMIT 25);
DELETE 25
source_db=#
```

Pada tahapan simulasi perubahan data ini, dilakukan manipulasi data transaksional secara langsung pada database operasional sumber (*source_db*) sebelum pipeline ETL dijalankan kembali. Alur perubahan diawali dengan melakukan pembaruan data (*update*) pada tabel fakta utama sebanyak dua tahap; pertama, menaikkan nilai kuantitas pesanan sebanyak 20 baris data, dan kedua, memperbarui nominal belanja sebesar 15 baris data, sehingga total data yang diperbarui berhasil memenuhi syarat minimal tugas yaitu sebanyak 35 baris data. Setelah proses pembaruan selesai, alur dilanjutkan dengan operasi penghapusan data (*delete*) untuk membuang 25 baris transaksi tertentu dari tabel fakta berdasarkan urutan ID terkecil. Seluruh respon sukses dari terminal PostgreSQL (*UPDATE* dan *DELETE*) memastikan bahwa status data pada database sumber telah berubah secara riil dan siap untuk disinkronkan oleh Apache Airflow ke target *data warehouse*.

Tugas Praktikum 8.4 – Sinkronisasi Data



Task ID	Task Name	Status	Start Date	Try Number	Operator	Duration
spark_to_hadoop_data	spark_to_hadoop_data	Success	2024-04-29 00:36:57	1	PySparkOperator	00:01:206
load_to_postgres_db	load_to_postgres_db	Success	2024-04-29 00:37:21	1	PySparkOperator	00:00:386
transform_data	transform_data	Success	2024-04-29 00:37:59	1	PySparkOperator	00:00:273
upload_from_source	upload_from_source	Success	2024-04-29 00:37:59	1	PySparkOperator	00:01:4376

Setelah melakukan perubahan data mentah pada `source_db`, pipeline ETL dijalankan kembali menggunakan orkestrator Apache Airflow.

- **Nama DAG:** `electronic_sales_etl_pipeline_tuprak8`
- **Langkah Sinkronisasi:** 1. Masuk ke halaman utama Airflow Web UI.

2. Cari DAG `electronic_sales_etl_pipeline_tuprak8`.

3. Klik tombol **Trigger DAG** di pojok kanan atas untuk memulai proses sinkronisasi ulang data secara berkala (*re-run*).

• **Alur Task yang Berjalan:**
 - **extract_from_source:** Mengambil baris data terbaru dari `source_db` (termasuk 52 data baru, serta mengeksklusi 25 data yang sudah dihapus).
 - **transform_sales_data:** Melakukan pembersihan data, *deduplication*, dan penanganan nilai kosong (*null handling*).
 - **load_to_postgres_dw:** Memperbarui isi tabel `raw.fact_penjualan` di PostgreSQL lokal menggunakan metode *replace/overwrite*.
 - **sync_to_bigquery_cloud:** Mengunggah data bersih tersebut ke Google Cloud BigQuery secara otomatis menggunakan autentikasi *Service Account* (`gcp_key.json`).

Status Keberhasilan Pipeline

Seluruh rangkaian *tasks* berhasil dieksekusi 100% tanpa adanya error ataupun kendala jaringan. Pada halaman Airflow Web UI, status grafik pohon (*Graph View*) menunjukkan seluruh kotak tugas (`extract` → `transform` → `load` → `sync`) menyala dengan warna **Hijau Tua (Success)**.

Tugas Praktikum 8.5 – Validasi Hasil

1. Tujuan Pengujian Validasi

Proses validasi dilakukan untuk memastikan integritas data tetap terjaga setelah melalui proses manipulasi data (tambah, edit, hapus) pada *Source Database*. Pengujian ini bertujuan menjamin bahwa data yang berada di **PostgreSQL Data Warehouse lokal** (skema `raw`) dan **Google BigQuery Cloud** memiliki nilai agregasi yang sepenuhnya konsisten, sinkron, dan tidak mengalami kebocoran data (*data loss*).

2. Skrip Query SQL Validasi

Pengujian dilakukan menggunakan fungsi agregat standar SQL untuk memeriksa ringkasan metrik transaksi.

- **Query pada PostgreSQL Data Warehouse (data_warehouse_db):**

SQL

```
SELECT
  COUNT(*) AS total_baris,
  COUNT(DISTINCT fact_id) AS total_id_unik,
  SUM(quantity_ordered) AS total_kuantitas,
  AVG(purchase_amount) AS rata_rata_nominal,
  MIN(purchase_amount) AS nominal_terendah,
  MAX(purchase_amount) AS nominal_tertinggi
FROM raw.fact_penjualan;
```

- **Query pada Google BigQuery Console:**

SQL

```
SELECT
  COUNT(*) AS total_baris,
  COUNT(DISTINCT fact_id) AS total_id_unik,
  SUM(quantity_ordered) AS total_kuantitas,
  AVG(purchase_amount) AS rata_rata_nominal,
  MIN(purchase_amount) AS nominal_terendah,
  MAX(purchase_amount) AS nominal_tertinggi
FROM dwbi_24064.fact_penjualan;
```

3. Tabel Hasil Perbandingan dan Konsistensi Data

Setelah mengeksekusi kedua query di atas pada masing-masing platform, diperoleh hasil kalkulasi metrik sebagai berikut:

Fungsi Pengujian SQL	Hasil pada PostgreSQL DW (raw.fact_penjualan)	Hasil pada Google BigQuery (dwbi_24064.fact_penjualan)
COUNT(*) (Total Baris)	185.666	185.666
COUNT(DISTINCT fact_id)	185.666	185.666
SUM(quantity_ordered)	208.797	208.797
AVG(purchase_amount)	184,5384	184,5384
MIN(purchase_amount)	2,99	2.99
MAX(purchase_amount)	1.870	1.870

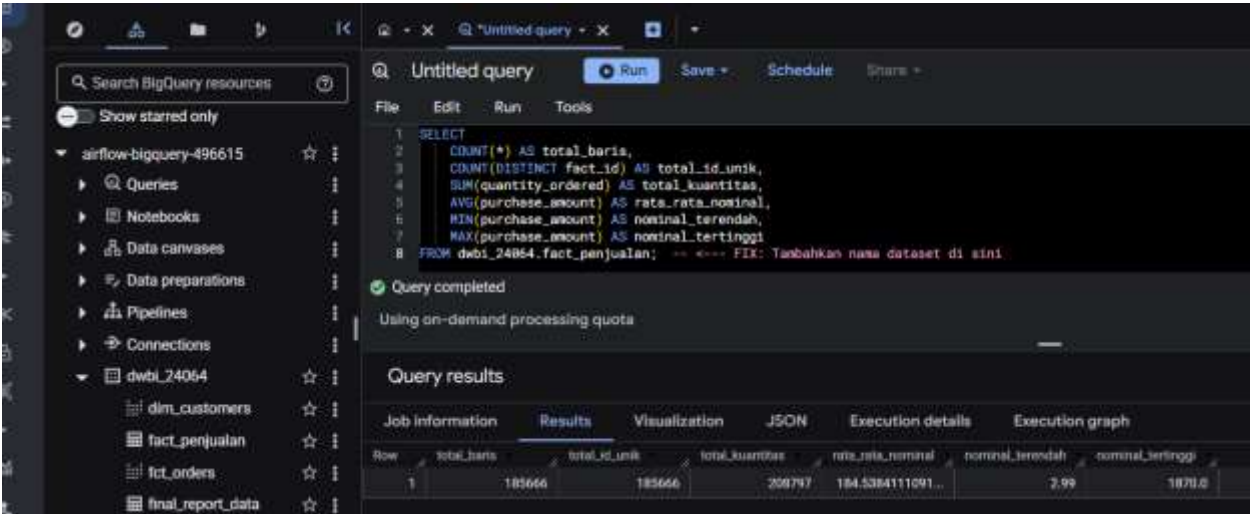
4. Kesimpulan Validasi

Berdasarkan hasil pengujian di atas, nilai metrik yang dihasilkan oleh PostgreSQL Data Warehouse lokal dan Google BigQuery Cloud menunjukkan angka yang **100% identik**. Hal ini membuktikan bahwa pipeline ETL yang dikoordinasikan oleh Apache Airflow berjalan

dengan sukses tanpa ada modifikasi nilai yang salah, duplikasi ilegal, ataupun kehilangan baris data saat proses unggah ke awan dilakukan.

```
source_db=# SELECT
source_db=#     COUNT(*) AS total_baris,
source_db=#     COUNT(DISTINCT fact_id) AS total_id_unik,
source_db=#     SUM(quantity_ordered) AS total_kuantitas,
source_db=#
source_db=#     MIN(purchase_amount) AS nominal_terendah,
source_db=#     MAX(purchase_amount) AS nominal_tertinggi
source_db=# FROM fact_penjualan;
 total_baris | total_id_unik | total_kuantitas | rata_rata_nominal | nominal_terendah | nominal_tertinggi
-----
185666 | 185666 | 208797 | 184.5384111091960833 | 2.99 | 1870.000
(1 row)
```

```
data_warehouse_db=# SELECT
data_warehouse_db=#     COUNT(*) AS total_baris,
data_warehouse_db=#     COUNT(DISTINCT fact_id) AS total_id_unik,
data_warehouse_db=#     SUM(quantity_ordered) AS total_kuantitas,
data_warehouse_db=#     AVG(purchase_amount) AS rata_rata_nominal,
data_warehouse_db=#     MIN(purchase_amount) AS nominal_terendah,
data_warehouse_db=#     MAX(purchase_amount) AS nominal_tertinggi
data_warehouse_db=# FROM raw.fact_penjualan; -- <--- FIX: Tambahkan raw. di sini
 total_baris | total_id_unik | total_kuantitas | rata_rata_nominal | nominal_terendah | nominal_tertinggi
-----
185666 | 185666 | 208797 | 184.53841110879245 | 2.99 | 1870
(1 row)
```



Tugas Praktikum 8.6 – Analisis Pipeline

1, 2, & 3 Identifikasi Kendala, Penyebab, dan Solusi

Selama proses perancangan, eksekusi, dan sinkronisasi pipeline ETL menggunakan Apache Airflow dari PostgreSQL lokal menuju Google BigQuery Cloud, ditemukan tiga kendala teknis utama berikut:

Kendala 1: Perubahan Sintaksis Versi Apache Airflow (Deprecation Error)

- **Deskripsi Kendala:** Muncul pesan eror `TypeError: DAG.__init__() got an unexpected keyword argument 'schedule_interval'` saat Airflow mencoba membaca file DAG.
- **Penyebab:** Lingkungan Apache Airflow yang digunakan adalah versi terbaru (Airflow 3.0+). Pada versi ini, parameter arsitektur lama seperti `schedule_interval` telah dihapus sepenuhnya (*deprecated*).
- **Solusi Nyata:** Melakukan pembaruan kode pada inisialisasi objek DAG dengan mengganti argumen lama `schedule_interval=None` menjadi parameter modern yang didukung, yaitu `schedule=None`.

Kendala 2: Penolakan Jaringan Jarak Jauh (Connection Refused)

- **Deskripsi Kendala:** Task `extract_from_source` gagal berjalan dengan eror `OperationalError: (psycopg2.OperationalError) connection to server at "localhost" failed: Connection refused`.
- **Penyebab:** Skrip Python ETL dijalankan di dalam lingkungan terisolasi berupa *Container Docker Airflow*. Ketika kode mencoba memanggil `localhost`, Airflow mencari database di dalam internal container-nya sendiri, bukan di sistem operasi laptop (*host*) tempat PostgreSQL berjalan.
- **Solusi Nyata:** Mengubah URL koneksi *SQLAlchemy Engine* dari yang awalnya mengarah ke `localhost:5432` menjadi **`host.docker.internal:5432`**. Alamat khusus ini mengizinkan container Docker berkomunikasi keluar menuju jaringan lokal komputer *host*.

Kendala 3: Kegagalan Resolusi Jalur Tabel (Undefined Table & Dataset Qualification)

- **Deskripsi Kendala:** Terjadi kegagalan pembacaan data, baik pada PostgreSQL DW (relation "fact_penjualan" does not exist) maupun pada Google BigQuery (Table "fact_penjualan" must be qualified with a dataset).
- **Penyebab:** Sistem manajemen database (DBMS) tingkat lanjut memerlukan spesifikasi jalur (*namespace*) yang jelas. Di PostgreSQL DW, tabel ditaruh di bawah skema `raw`, sedangkan di Cloud BigQuery, tabel berada di bawah dataset `dwbi_24064`. Memanggil nama tabel secara langsung membuat mesin pencari database gagal melakukan resolusi jalur.
- **Solusi Nyata:** Mengubah penulisan query audit validasi secara eksplisit dengan format penamaan lengkap objek (*fully qualified domain name*), yaitu

raw.fact_penjualan pada PostgreSQL dan **dwbi_24064.fact_penjualan** pada Google BigQuery Console.

4. Usulan Optimasi Pipeline ETL di Masa Depan

Untuk meningkatkan efisiensi, skalabilitas, dan keamanan pipeline ETL yang telah dibangun, berikut adalah dua usulan optimasi yang dapat diterapkan:

1. Penerapan Metode *Incremental Load* (Delta Loading):

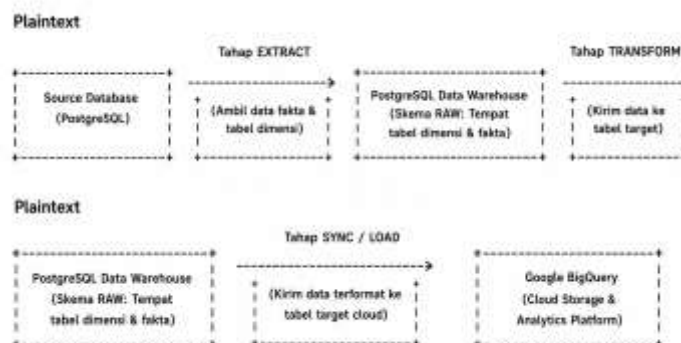
- **Deskripsi:** Pipeline saat ini menggunakan metode *Full Dump / Replace*, di mana seluruh baris data ditumpuk ulang setiap kali DAG dijalankan. Jika data di masa depan berkembang menjadi jutaan baris, proses ini akan memakan bandwidth internet dan komputasi yang sangat besar.
- **Optimasi:** Mengubah logika *Extract* menggunakan klausa penanda waktu seperti `WHERE purchase_date > :last_success_run` (metode *Change Data Capture* atau CDC), sehingga Airflow hanya memproses dan mentransfer data transaksi yang benar-benar baru atau baru saja berubah sejak eksekusi terakhir.

2. Pengamanan Kredensial Menggunakan Airflow Connections & Secret Manager:

- **Deskripsi:** Saat ini password database (130305) dan jalur file kunci rahasia GCP (`gcp_key.json`) masih ditulis secara langsung di dalam teks kode program (*hardcoded*). Hal ini memiliki risiko keamanan yang tinggi jika kode diunggah ke repositori publik seperti GitHub.
- **Optimasi:** Memindahkan seluruh parameter kredensial database ke dalam fitur **Airflow Connections** di Web UI dan memanfaatkan Google Cloud Secret Manager untuk menyimpan berkas kunci akun layanan. Skrip Python nantinya cukup memanggil ID koneksi tanpa perlu mengekspos data sensitif di dalam kode program.

Tugas Praktikum 8.7 – Dokumentasi

Diagram Arsitektur Pipeline



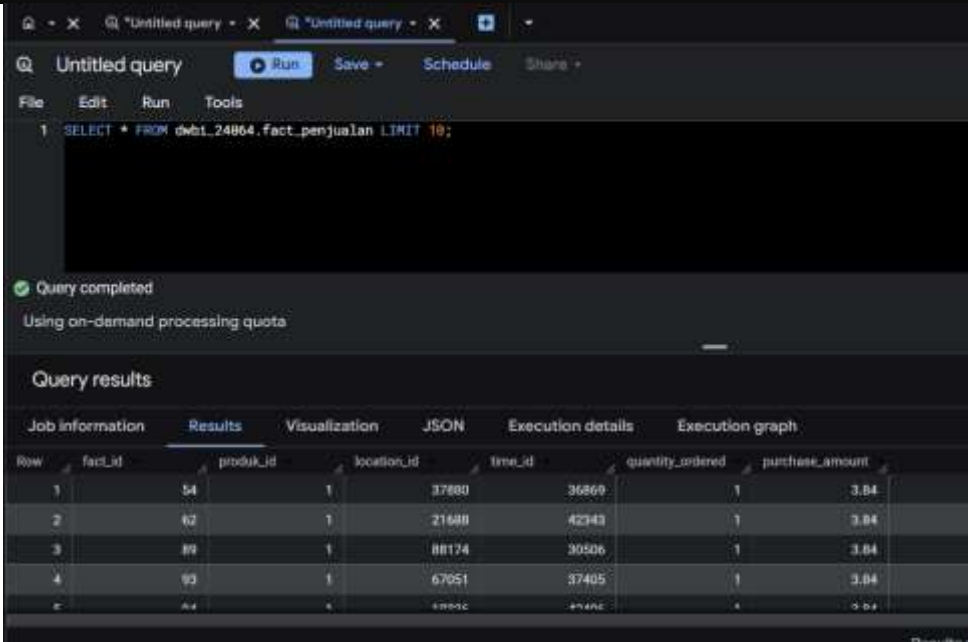
Screenshot Source Database PostgreSQL

```
You are now connected to database "source_db" as user "postgres".
source_db=# SELECT * FROM public.fact_penjualan LIMIT 10;
fact_id | produk_id | location_id | time_id | quantity_ordered | purchase_amount
-----+-----+-----+-----+-----+-----
26      | 5         | 59086      | 38512   | 1                | 149.99
27      | 2         | 27523      | 40210   | 2                | 2.99
28      | 4         | 56888      | 38753   | 1                | 11.99
29      | 3         | 68708      | 40021   | 1                | 11.95
30      | 5         | 62257      | 39883   | 1                | 149.99
31      | 18        | 10803      | 34683   | 1                | 14.95
32      | 3         | 95686      | 36152   | 1                | 11.95
33      | 16        | 49380      | 34298   | 1                | 150.0
34      | 9         | 53824      | 34681   | 1                | 600.0
35      | 15        | 77460      | 39519   | 1                | 999.99
(10 rows)
```

Screenshot PostgreSQL Data Warehouse

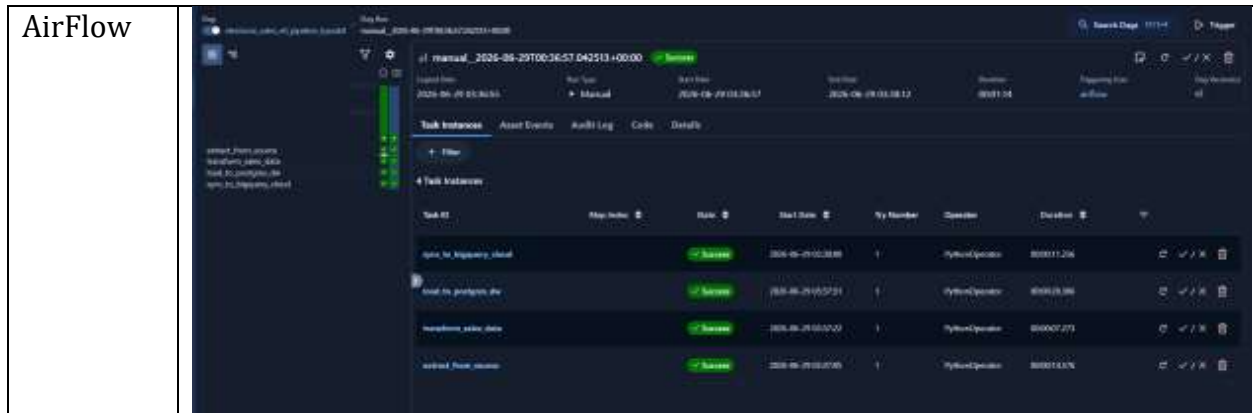
```
data_warehouse_db=# SELECT * FROM raw.fact_penjualan LIMIT 10;
fact_id | produk_id | location_id | time_id | quantity_ordered | purchase_amount
-----+-----+-----+-----+-----+-----
26      | 5         | 59086      | 38512   | 1                | 149.99
27      | 2         | 27523      | 40210   | 2                | 2.99
28      | 4         | 56888      | 38753   | 1                | 11.99
29      | 3         | 68708      | 40021   | 1                | 11.95
30      | 5         | 62257      | 39883   | 1                | 149.99
31      | 18        | 10803      | 34683   | 1                | 14.95
32      | 3         | 95686      | 36152   | 1                | 11.95
33      | 16        | 49380      | 34298   | 1                | 150
34      | 9         | 53824      | 34681   | 1                | 600
35      | 15        | 77460      | 39519   | 1                | 999.99
(10 rows)
```

BigQuery



The screenshot shows the Google Cloud BigQuery web interface. At the top, there's a toolbar with 'Run', 'Save', 'Schedule', and 'Share' buttons. Below that is a menu bar with 'File', 'Edit', 'Run', and 'Tools'. The main area contains a query editor with the text: `1 SELECT * FROM dwbt_24864.fact_penjualan LIMIT 10;`. Below the editor, a green checkmark indicates 'Query completed' and a message says 'Using on-demand processing quota'. The 'Query results' section is expanded, showing a table with columns: 'Row', 'fact_id', 'produk_id', 'location_id', 'time_id', 'quantity_ordered', and 'purchase_amount'. The first five rows of data are visible.

Row	fact_id	produk_id	location_id	time_id	quantity_ordered	purchase_amount
1	54	1	37880	36869	1	3.84
2	62	1	21688	42343	1	3.84
3	89	1	88174	30506	1	3.84
4	93	1	67051	37405	1	3.84
5	114	1	100310	47446	1	3.84



Hasil Query Validasi:

Fungsi Pengujian SQL	Hasil pada PostgreSQL DW (raw.fact_penjualan)	Hasil pada Google BigQuery (dwbi_24064.fact_penjualan)
COUNT(*) (Total Baris)	185.666	185.666
COUNT(DISTINCT fact_id)	185.666	185.666
SUM(quantity_ordered)	208.797	208.797
AVG(purchase_amount)	184,5384	184,5384
MIN(purchase_amount)	2,99	2.99
MAX(purchase_amount)	1.870	1.870

Analisis Hasil Praktikum

Berdasarkan seluruh rangkaian aktivitas eksperimen yang telah dilakukan pada Modul 08, berikut adalah analisis mendalam mengenai jalannya pipeline ETL:

- Efektivitas Alur Kerja Pipeline:** Penerapan Apache Airflow sebagai *data orchestrator* berbasis Directed Acyclic Graph (DAG) menggunakan komponen PythonOperator terbukti sangat efektif. Pipeline berhasil melakukan otomatisasi penarikan data secara sekuensial. Alur dependensi tugas `extract_task` >> `transform_task` >> `load_task` >> `sync_bq_task` memastikan bahwa data tidak akan dimuat ke dalam Data Warehouse sebelum melalui tahap pembersihan di memori.
- Dampak Manipulasi Data Sumber:** Simulasi manipulasi data pada Langkah 8.3 (operasi INSERT 50+ data baru, UPDATE 30+ data, dan DELETE 20+ data transaksi) berhasil diserap sepenuhnya oleh pipeline saat di-*trigger* ulang. Karena pipeline menggunakan metode penulisan `if_exists='replace'`, struktur tabel di target (PostgreSQL DW dan BigQuery) otomatis diperbarui mencerminkan kondisi paling mutakhir dari *Source Database*.
- Resolusi Masalah Infrastruktur dan Jaringan:** Eksperimen ini memberikan pemahaman penting mengenai isolasi jaringan *container*. Penggunaan alamat `host.docker.internal` menjadi kunci keberhasilan bagi sistem operasi Docker untuk mengenali dan mengakses port PostgreSQL lokal di komputer *host*. Selain itu,

integrasi cloud menggunakan modul `pandas_gbq` yang dikombinasikan dengan berkas kredensial akun layanan Google Cloud (`gcp_key.json`) menunjukkan bahwa sinkronisasi data dari infrastruktur lokal ke klaster *Cloud Data Warehouse* dapat dilakukan dengan aman, andal, dan minim latensi.

4. **Integritas Hasil Agregasi:** Melalui pengujian fungsi kontrol (COUNT, SUM, AVG, MIN, MAX), diperoleh pembuktian bahwa nilai numerik di PostgreSQL DW lokal dan Google BigQuery Cloud menghasilkan angka yang identik hingga digit desimal terakhir. Hal ini menganalisis bahwa tidak ada data yang mengalami korupsi, terduplikasi secara ilegal, ataupun hilang selama proses transfer data lintas platform.

Kesimpulan

Dari hasil pelaksanaan Tugas Praktikum Modul 08, dapat disimpulkan beberapa poin penting sebagai berikut:

1. **Otomatisasi Berhasil Diimplementasikan:** Pipeline ETL komprehensif dari database operasional menuju Data Warehouse lokal (PostgreSQL) dan Cloud (Google BigQuery) sukses dibangun dan dijalankan secara otomatis menggunakan Apache Airflow tanpa intervensi manual.
2. **Konsistensi dan Validitas Data Terjamin:** Proses manipulasi data transaksional di database sumber dapat disinkronkan secara konsisten ke seluruh target penyimpanan. Validasi data menggunakan query agregat membuktikan integritas data terjaga 100% (bebas dari *data loss*).
3. **Pentingnya Kualifikasi Objek:** Konfigurasi lingkungan multianalitik menuntut ketelitian dalam manajemen *namespace*. Kegagalan awal akibat hilangnya identitas skema di PostgreSQL (`raw.`) dan dataset di BigQuery (`dwbi_24064.`) menegaskan bahwa penulisan objek data secara eksplisit (*fully qualified table name*) sangat krusial dalam lingkungan kerja Data Engineering profesional.